

Программирование на Java

Введение

Юрий Литвинов
y.litvinov@spbu.ru

19.09.2026



Формальности

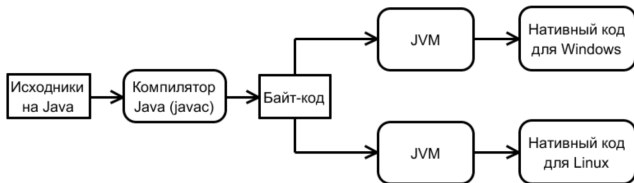
- ▶ Практический курс, в формате мастер-классов и командной работы на паре
- ▶ Будет несколько домашних заданий, две «контрольные» и пара с докладами
 - ▶ По домашним работам будут дедлайны, -1 балл за каждую неделю просрочки, начиная с нулевой
 - ▶ Контрольные в форме устной беседы
 - ▶ Баллы за домашние и контрольные приводятся к диапазону 0-100, берётся минимум
 - ▶ К нему прибавляются баллы за работу в аудитории и за доклады
- ▶ Условия, материалы и сдача домашек через <https://hwproj.ru/courses/50105>
- ▶ Среда программирования — какая угодно
 - ▶ OpenIDE, GigaIDE, VS Code, IntelliJ IDEA (если сможете её достать)
- ▶ Преподаватели: Юрий Литвинов (y.litvinov@spbu.ru), Юрий Уфимцев (YuriUfimtsev@yandex.ru)

Что будет в курсе

- ▶ Особенности синтаксиса Java
- ▶ Модульное тестирование в Java (JUnit, Mockito, Hamcrest)
- ▶ Коллекции стандартной библиотеки
- ▶ Stream API
- ▶ Работа с базами данных (JDBC, JPA, Hibernate, Spring Data JPA)
- ▶ Веб-сервисы и веб-приложения, Spring
- ▶ Веб-API — REST, gRPC, GraphQL
- ▶ Очереди сообщений, Apache Kafka, RabbitMQ
- ▶ Многопоточное программирование — synchronized, Future, пул потоков

Язык Java

- ▶ Появился в 1995 году, актуальная версия — Java 27
 - ▶ Актуальная LTS-версия — Java 25
 - ▶ Всё ещё популярна Java 8
- ▶ Объектно-ориентированный язык с сильной типизацией
- ▶ Прежде всего — для разработки прикладного программного обеспечения
 - ▶ Большая часть современного Enterprise — на Java
- ▶ Использует виртуальную машину (compile once — run everywhere)
- ▶ Just-in-time-компиляция с довольно продвинутым рантаймом
- ▶ Со всем этим вы должны быть знакомы по Kotlin, потому что он использует инфраструктуру Java

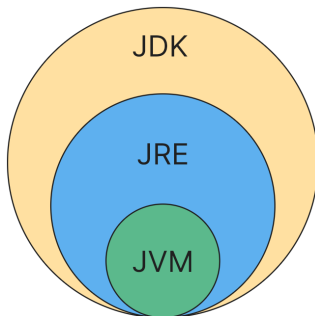


Особенности

- ▶ Язык создавался на пике развития ООП, сам сильно повлиял на методологию разработки
 - ▶ «Настоящий» теоретически правильный объектно-ориентированный язык
 - ▶ Создавался на базе опыта C++ с учётом его ошибок (и Smalltalk)
 - ▶ C# появился на пять лет позднее и учёл ряд ошибок уже Java
- ▶ Практически всё — объект, иногда даже неожиданно: например, enum-ы — это полноценные классы с состоянием и методами
- ▶ Все методы по умолчанию виртуальные, нет свободных функций
- ▶ Много информации о типах известно во время выполнения (развитая рефлексия), расширяемый компилятор
- ▶ Несколько странная реализация генериков
- ▶ Много разных реализаций виртуальной машины

Странные аббревиатуры

- ▶ JVM (Java Virtual Machine) — то, что исполняет байт-код
 - ▶ JIT-компилятор, сборщик мусора и т.п.
- ▶ JRE (Java Runtime Environment) — среда исполнения (JVM + набор библиотек)
- ▶ JDK (Java Development Kit) — набор разработчика
 - ▶ Компилятор, утилиты для работы с JAR, jshell, javadoc и т.п.



Реализации JDK

- ▶ OpenJDK — реализация с открытым исходным кодом от Oracle
 - ▶ Что-то вроде референсной реализации, не очень для продуктового окружения
- ▶ Oracle JDK — официальный JDK от Oracle, платный для коммерческого использования
- ▶ Eclipse Temurin — бесплатная вендор-нейтральная реализация
- ▶ Azul Zulu — неплохой коммерческий дистрибутив
- ▶ Axiom JDK, SberJDK — российские дистрибутивы на базе OpenJDK

OpenJDK



Какие ещё языки есть на платформе Java

- ▶ Kotlin — «Java done right», официальный язык для разработки под Android и неплохой выбор для Enterprise-разработки
- ▶ Scala — в основном функциональный язык, для сурового бэкенда
 - ▶ Не стал суперпопулярен, хотя вовсе не эзотерика — сложный
- ▶ Groovy — в основном для мелких скриптов (например, систем сборки)
- ▶ Clojure — диалект LISP для JVM (функциональный)
 - ▶ Скорее эзотерика
- ▶ Многие другие



Mandatory slide про стандартные числовые типы

Тип	Значения	Размер
<i>byte</i>	$-2^7..2^7 - 1$ ($-128..127$)	8 бит
<i>short</i>	$-2^{15}..2^{15} - 1$ ($-32768..32767$)	16 бит
<i>int</i>	$-2^{31}..2^{31} - 1$	32 бит
<i>long</i>	$-2^{63}..2^{63} - 1$	64 бит
<i>float</i>	$-3.4028235E+38..-1.4E-45$ и $1.4E-45..3.4028235E+38$	32 бит
<i>double</i>	$-1.7976931348623157E+308..-4.9E-324$ и $4.9E-324..1.7976931348623157E+308$	64 бит

Ссылочные типы и типы-значения

- ▶ Примитивные типы:
byte, short, int, long, float, double, boolean, char
- ▶ Ссылочные типы: массивы, классы (в том числе строки и типы-обёртки), интерфейсы, перечисления, аннотации
- ▶ Ссылочные типы всегда¹ хранятся на куче и передаются по ссылке (точнее, ссылка передаётся по значению), примитивные типы всегда хранятся и передаются по значению
- ▶ У каждого типа есть значение по умолчанию — **null** для ссылочных типов (в том числе массивов и строк), нули для всех остальных
- ▶ Оператор == для ссылочных типов всегда сравнивает их место в памяти²
 - ▶ Строки нельзя сравнивать ==, используйте equals

¹ Не совсем всегда, компилятор умеет сам иногда размещать объекты на стеке, если это не ломает семантику

² Технически это не совсем правда, == сравнивает идентичность объектов, с учётом их перемещений в памяти при сборке мусора и т.п.

Типы-обёртки, что?

- ▶ Классы, соответствующие примитивным типам и поддерживаемые компилятором
- ▶ Boxing/Unboxing
- ▶ **byte** — Byte, **short** — Short, ну вы поняли
- ▶ Не всё так просто: **char** — Character, **int** — Integer
- ▶ Методы: **int** ololo = Integer.parseInt("239");
- ▶ Известная «особенность»:
 - ▶ Integer.valueOf(127) == Integer.valueOf(127)
 - ▶ но Integer.valueOf(128) != Integer.valueOf(128)

OpenIDE, демонстрация

Демонстрация

Что скачать и поставить

- ▶ JDK 27 (<https://jdk.java.net/27/>)
 - ▶ Обратите внимание, JRE — среда времени выполнения, JDK — среда времени выполнения + инструменты разработки (включая компилятор)
 - ▶ Очень желательно добавить `javac` в `PATH` и прописать переменную окружения `JAVA_HOME`
- ▶ OpenIDE (<https://openide.ru/>) или VS Code

Что сдавать в домашках

- ▶ Файлы .java
- ▶ Папку .idea
 - ▶ Кроме workspace.xml, usage.statistics.xml, tasks.xml
- ▶ Файлы .iml (если есть)
- ▶ Ничего больше

Как собирать из консоли

▶ javac

- ▶ `javac MyClass.java YetAnotherClass.java`
- ▶ `javac -d classes MyClass.java`
- ▶ `javac -classpath classes;library.jar -d classes MyClass.java`

▶ CLASSPATH

- ▶ Набор путей, по которым компилятор и Java-машина ищут классы
- ▶ Всегда содержит классы из стандартной библиотеки
- ▶ По умолчанию — текущая директория (“.”)
- ▶ Задаётся как список директорий или JAR-файлов, через “;” в Windows и через “:” во всём остальном
 - ▶ JAR-файл — просто заархивированная папка с классами, по сути — библиотека

▶ Системы сборки — Gradle, Maven, ...

Как запускать из консоли

- ▶ Нет никакого .exe-шника, виртуальной машине передаётся на исполнение файл с байт-кодом класса
 - ▶ При желании его можно собрать, но надо специально озаботиться
- ▶ Оный класс должен иметь метод `public static void main(String[] args)`
- ▶ `java (javaw)`
 - ▶ `java MyClass`
 - ▶ `java -classpath classes_dir;library.jar MyClass`
 - ▶ `java -jar library_with_main_class.jar`
 - ▶ не всё так просто, jar-нику нужен манифест
 - ▶ Имя запускаемого класса должно быть **полностью квалифицированным**
 - ▶ например, `java com.example.MyClass`
 - ▶ Нелишне посмотреть документацию, есть много полезных ключей командной строки
- ▶ Системы сборки сильно облегчают эту боль

Некоторые тонкости IntelliJ Platform

- ▶ Есть отдельно меню File -> Settings и отдельно File -> Project Structure
 - ▶ Settings — это в основном настройки самой среды
 - ▶ Project Structure — это настройки проекта
 - ▶ Версия языка (есть отдельно версия языка и отдельно версия SDK)
 - ▶ Модули — в какой папке код, в какой тесты, в какой ресурсы; IntelliJ компилирует только папки, отмеченные как Sources или Tests
- ▶ Справа сверху — конфигурации запуска. Там можно настроить, например, параметры командной строки
- ▶ Знание основных хоткеев сильно ускоряет разработку

Стайлгайд

- ▶ <https://google.github.io/styleguide/javaguide.html>
- ▶ На что обратить внимание:
 - ▶ camelCase для методов и «переменных», CamelCase для типов, КАПС_ДЛЯ_КОНСТАНТ
 - ▶ Правильные имена пакетов (DNS-имя наоборот + собственно имя пакета)
 - ▶ «Египетские» фигурные скобки (так же известны, как K & R)
 - ▶ Минимально возможная видимость полей, методов и всего-всего
 - ▶ Всегда указывайте модификатор видимости
 - ▶ Комментарии к каждому классу и каждому public-методу
 - ▶ JavaDoc

JavaDoc

- ▶ Стандартная система генерации документации
- ▶ В IntelliJ Platform это Tools -> Generate JavaDoc
- ▶ `/** */` — JavaDoc-комментарий
- ▶ Сначала общее описание, затем, опционально, уточняющие тэги
- ▶ Тэги:
 - ▶ `@param` имя описание параметра
 - ▶ `@return` описание возвращаемого значения
 - ▶ `@exception` ИмяКлассаИсключения описание, когда бросается
 - ▶ `@inheritDoc`
 - ▶ `@see`
- ▶ Пустые тэги не очень полезны